学案 02 | 后续会用到的数学直觉

本学案不要求高等数学证明。目标是能看懂输入和输出的形状，知道概率如何归一化、误差如何变成参数更新方向。

1. 标量、向量和矩阵

- **标量（scalar）**：一个数，例如损失 `0.7`。
- **向量（vector）**：一列或一行数，例如 `[0.2, 0.5, -0.1]`。
- **矩阵（matrix）**：按行列排布的数，例如 `[[1, 2], [3, 4]]`。
- **张量（tensor）**：PyTorch 中多维数组的统称；标量、向量和矩阵也都可以视为不同维度的张量。

形状（shape）描述每个维度有多少个元素。比如形状 `(2, 3)` 表示 2 行 3 列；形状 `(4, 2, 3)` 表示三维数组，三维长度分别是 4、2、3。形状用来检查操作是否符合预期，不要只看打印出来的数值。

一个矩阵乘法小例子

```
输入 x = [2, 3]              形状 (2,)
权重 W = [[1, 0],            形状 (2, 2)
          [0, 1]]
输出 y = xW = [2, 3]          形状 (2,)
```

这里 W 是单位矩阵，所以输出和输入相同。矩阵乘法的一项重要用途，是用可学习权重对输入数值做组合变换。大模型反复使用大量矩阵运算；阶段 1 再看这些运算如何组成注意力和前馈层。

做题时先检查维度能否相乘。若 `x` 有 2 个数，而 `W` 的行数为 2，则可以组合出结果；具体库的行/列记法可能不同，必须根据代码的张量形状确认。

2. 预测分数如何变成概率

对一组 logits 应用 softmax 会将分数转换成概率分布。直觉上，较高的 logit 会得到较大的概率；最终概率都在 0–1 之间并且加总为 1。

例如三个类别的概率是：

```
猫：0.70
狗：0.20
鸟：0.10
总和：1.00
```

如果正确答案是“猫”，模型给正确类别的概率较高；如果正确答案是“鸟”，虽然其他类别更常见，但本例中模型对正确类别的信心低。这正是交叉熵损失会惩罚的情形。

3. 损失：把训练目标变成可比较的数

损失函数（loss function）把预测和训练目标映射成一个数。数值大小表示在当前损失定义下，预测与目标有多不一致。优化器试图减小该数值。

均方误差的直觉

对一个数值预测，均方误差（MSE）可以写成：

```
预测值：2.0
目标值：3.0
误差平方：(2.0 − 3.0)² = 1.0
```

误差符号被平方消去，偏差更大的预测会受到更大的惩罚。MSE 常用于回归示例；它不是所有任务的默认损失。

交叉熵的直觉

对多类别预测，交叉熵关注模型给**正确类别**分配了多少概率。直觉公式是“正确类别概率越低，损失越高”，可写成 `-log(正确类别概率)`。例如正确类别概率从 `0.8` 降到 `0.1`，损失会明显变大。

语言模型训练常见目标是预测下一个 token，并对正确 token 计算交叉熵。损失只说明该目标上的拟合情况，不能单独证明事实性、安全性、推理能力或 Agent 任务表现。

4. 梯度：误差对参数的方向提示

梯度（gradient）描述某个量改变时，损失会怎样变化。对一个参数来说：

- 梯度为正：在当前位置把参数稍微增大，损失倾向于变大。
- 梯度为负：在当前位置把参数稍微增大，损失倾向于变小。
- 梯度绝对值较大：附近的损失对参数变化较敏感。

梯度下降（gradient descent）通常沿梯度的相反方向移动参数：

```
新参数 = 旧参数 − 学习率 × 梯度
```

学习率（learning rate）控制每次更新步幅。步幅太大可能越过较好的区域、训练不稳定；太小则学习缓慢。真实神经网络参数很多，梯度由自动微分程序计算，不需要阶段 0 手算每一个参数。

5. 一个手算更新

用非常小的模型预测 `y = wx + b`。当前 `x=1`，正确目标 `y=3`，初始 `w=1, b=0`，预测是 1。采用单样本平方损失：

```
L = (预测 − 目标)² = (1 − 3)² = 4
```

在这个点，`w` 和 `b` 的梯度都为 `-4`。若学习率是 `0.1`，更新后 `w=1.4, b=0.4`，新预测为 `1.8`，损失为 `(1.8-3)²=1.44`。这一步让模型更接近目标，但还没完全学会关系。真实训练需要多个样本和多次更新。

关键认识：训练不是“损失告诉模型正确答案是什么”。样本目标提供方向，梯度描述如何改变参数使目标损失下降。

6. 小批量与验证集

一次用一组样本估计损失和梯度，叫一个 **batch（小批量）**。把训练样本分成多个 batch 逐步更新，能控制内存并稳定计算。遍历一遍训练集常叫一个 epoch。

数据通常至少区分：

- **训练集（training set）**：用于计算训练信号、更新参数。
- **验证集（validation set）**：训练中用于观察泛化和选择设置，不用于那一轮梯度更新。
- **测试集（test set）**：最终评估时使用，尽量不反复据此调参。

如果模型只在训练数据上越来越好，在未参与训练的数据上变差，可能出现过拟合（overfitting）。这说明“训练损失下降”不等于“对新情况能力提高”。

7. 快速自检

1. 形状 `(3, 4)` 表示什么？它是 3 个元素还是 12 个元素？
2. 正确类别的预测概率很低，交叉熵通常偏大还是偏小？
3. 若某个参数的梯度为负，梯度下降会把它往哪个方向更新？
4. 为什么验证集不应该被反复用于梯度更新？
5. 训练损失下降但验证损失上升，可能意味着什么？